21CSC558J DEEP LEARNING APPROACHES

Module-2 - Optimization, Regularization and Normalization in Deep Neural Networks

Session-03 & Session-04

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Module-2 - Optimization, Regularization and Normalization in Deep Neural Networks

- Gradient descent algorithm, Types of gradient descent, RMS prop, Adagrad, ADAM optimiser, Unit Saturation-
- Vanishing And Exploding Gradient, Underfitting, Overfitting And Learning Rate,
- Regularization Techniques In Neural Networks- LASSO Regression, Ridge Regression, Dropouts, Early Stopping
- Normalization techniques in Neural Networks - Batch Normalization, Group Normalization, Instance Normalization
- Lab 3: Building neural network using Keras
- Lab 4: Building programs to optimize the neural network using gradient descent. normalization techniques

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Session-03

- 3.1 Gradient descent algorithm, Types of gradient descent, RMS prop, Adagrad, ADAM optimiser, Unit Saturation
- 3.2 Vanishing And Exploding Gradient, Underfitting, Overfitting and Learning Rate

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Gradient descent algorithm

Gradient Descent is an optimization algorithm used to **minimize loss function** by updating model parameters (weights and bias).

Gradient descent is a mathematical technique that iteratively finds the weights and bias that produce the model with the lowest loss.

Gradient descent finds the best weight and bias by repeating the following process for a number of user-defined iterations.

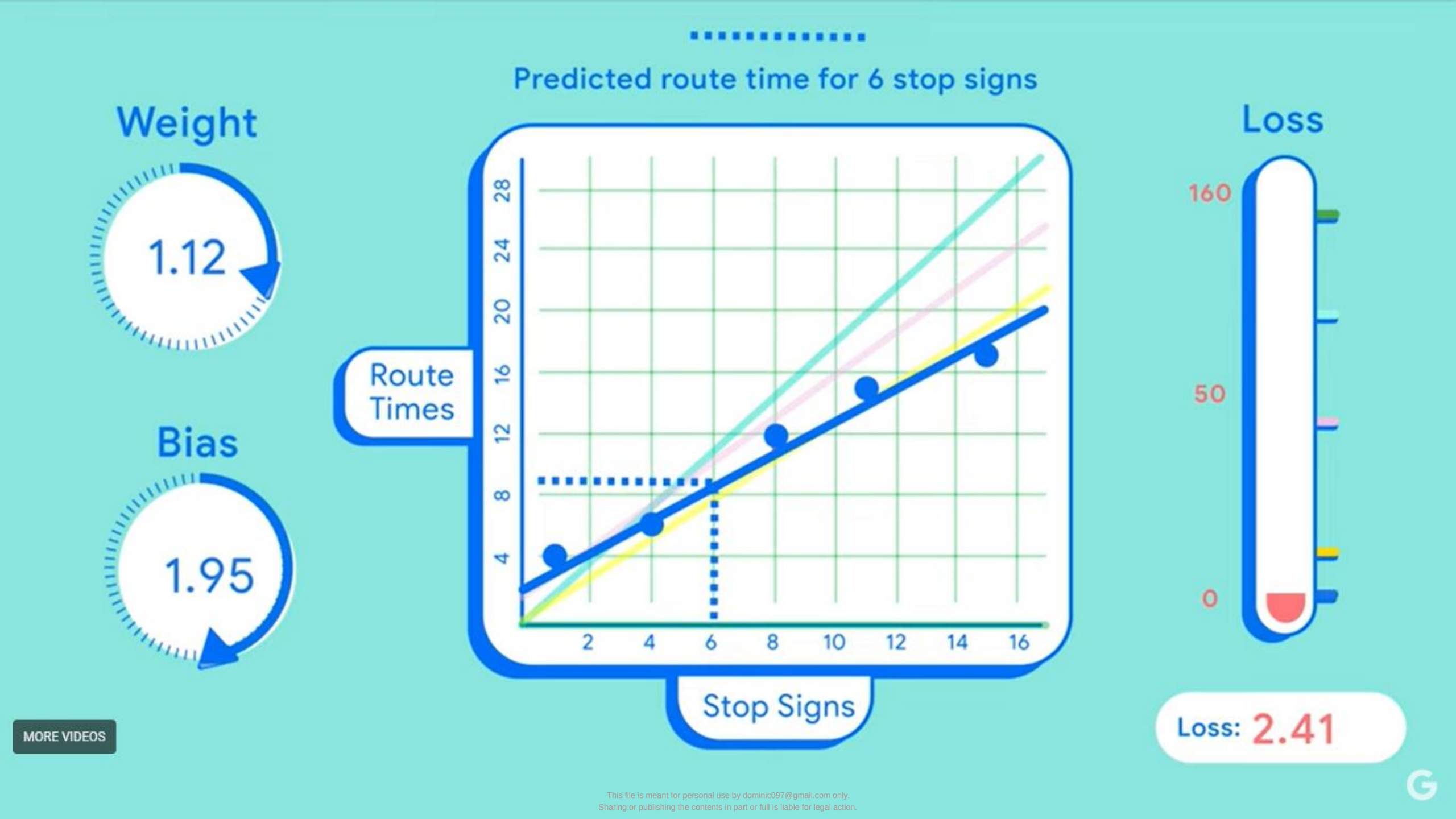
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Gradient descent algorithm

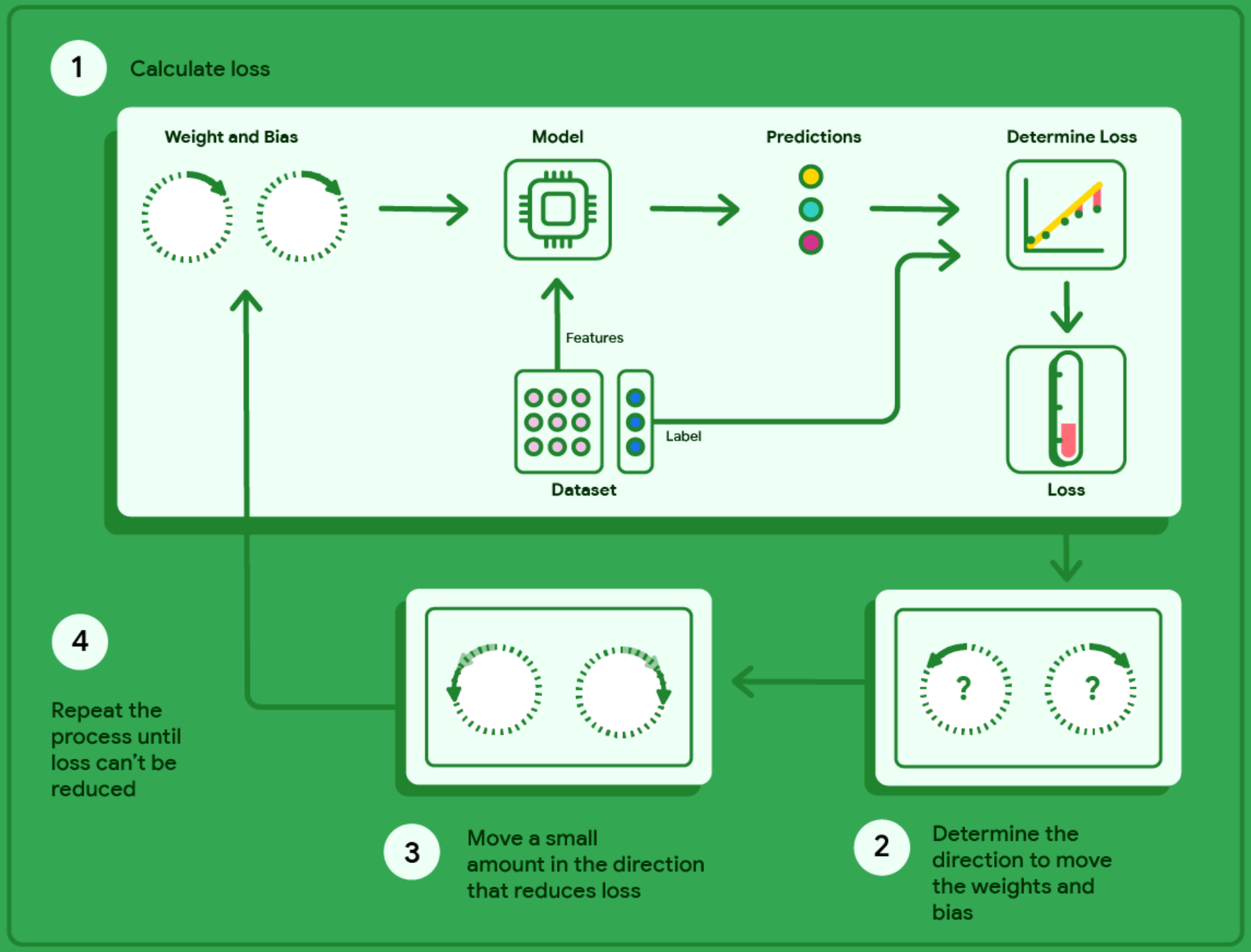
- The model begins training with randomized weights and biases near zero
- Repeats the following steps
  - Calculate the loss with the current weight and bias.
  - Determine the direction to move the weights and bias that reduce loss.
  - Move the weight and bias values a small amount in the direction that reduces loss.
  - Return to step one and repeat the process until the model can't reduce the loss any further.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



This file is meant for personal use by dominic297@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Gradient-Based Optimization Algorithms

- Gradient-based optimization algorithms are used in Machine Learning and Deep Learning to **minimize loss functions** by iteratively updating model parameters using gradients.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is Gradient Descent ?

- It updates weights in the direction of **negative gradient (steepest descent)** of the loss function.
- Update Rule:

$$w = w - \alpha \nabla L(w)$$

- Where:
  - w→ Model parameters
  - α→ Learning rate
  - $L(w)$ →Loss function

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Basic Gradient Descent

```
import numpy as np

# Simple function: f(x) = x^2
def f(x):
    return x**2

def grad(x):
    return 2*x

x = 5  # initial value
lr = 0.1

for i in range(20):
    x = x - lr * grad(x)
    print(f"Step {i}, x = {x}, f(x) = {f(x)}")
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Types of Gradient Descent

Basic Gradient Descent Algorithms

- Batch gradient descent
- Stochastic gradient descent
- Mini-batch gradient descent

Adaptive Learning Rate Optimizers

- Adagrad
- RMSProp
- Adam

Momentum-Based Optimizers

- Momentum Gradient Descent
- Nesterov Accelerated Gradient (NAG)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch Gradient Descent

Uses the **entire training dataset** to compute gradient for each update.

For dataset of size N:

- Compute gradient using all N samples
- Update weights once

Advantages

- Stable convergence
- Smooth optimization path

Disadvantages

- Very slow for large datasets
- Requires high memory

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example

Algorithm Steps

1. Initialize weights randomly
2. Compute prediction for all samples
3. Calculate total loss
4. Compute gradient using full dataset
5. Update weights
6. Repeat until convergence

```
import numpy as np

X = np.array([1,2,3,4])
y = np.array([2,4,6,8])

w = 0
lr = 0.01

for epoch in range(100):
    y_pred = w*X
    grad = np.mean(2*(y_pred - y)*X)
    w = w - lr*grad

print("Weight:", w)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 1 – Medical Research Model Training

- Scenario
  - Training cancer prediction model
  - Dataset = 5,000 patient records
  - Training done offline in lab
- Why Batch GD?
  - Dataset manageable
  - Accuracy more important than speed
  - Stable learning needed

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 2 – Financial Risk Modeling

- Scenario
  - Bank credit risk model updated monthly
  - Uses full historical customer dataset
- Why Batch GD?
  - Needs consistent predictions
  - No real-time urgency

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch GD

Model waits for **entire dataset** → then updates weights once.

Study **entire syllabus**, then revise once.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Stochastic Gradient Descent (SGD)

Uses **one training sample at a time** to update weights.

For dataset size N:

- Update weights N times per epoch

Advantages

- Faster updates
- Works well for very large datasets
- Helps escape local minima

Disadvantages

- Noisy convergence
- Less stable

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example

Algorithm Steps

For each epoch:

- Shuffle dataset
- For each sample:
  - Compute loss
  - Compute gradient
  - Update weight immediately

```
import numpy as np

X = np.array([1,2,3,4])
y = np.array([2,4,6,8])

w = 0
lr = 0.01

for epoch in range(50):
    for i in range(len(X)):
        y_pred = w*X[i]
        grad = 2*(y_pred - y[i])*X[i]
        w = w - lr*grad

print("Weight:", w)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 1 – Online Recommendation System

- Scenario
  - User clicks product → model updates immediately
  - Used in e-commerce personalization
- Why SGD?
  - Real-time adaptation
  - Continuous learning
  - Handles streaming data

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 2 – Fraud Detection in Transactions

- Scenario
  - Every new transaction updates fraud model
  - Fraud patterns change quickly
- Why SGD?
  - Fast response
  - Learns new fraud patterns instantly

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 3 – Speech Recognition Personalization

- Scenario
  - Voice assistant improves based on user corrections
- Why SGD?
  - Learns from each interaction

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

SDG

Model updates **after every single data sample.**

Learn from **each question immediately.**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Mini-Batch Gradient Descent

- Mini-Batch Gradient Descent updates weights using **small subset of dataset**.
- Typical Batch Sizes:
  - 16
  - 32
  - 64
  - 128

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example

```
import numpy as np
X = np.random.rand(100)
y = 3*X + 2

w = 0
lr = 0.01
batch_size = 10

for epoch in range(50):
    for i in range(0, len(X), batch_size):
        X_batch = X[i:i+batch_size]
        y_batch = y[i:i+batch_size]

        y_pred = w*X_batch
        grad = np.mean(2*(y_pred - y_batch)*X_batch)
        w = w - lr*grad

print("Weight:", w)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 1 – Self-Driving Car Vision Training

- Scenario
  - Train model using camera images
  - Images processed in batches of 64
- Why Mini-Batch?
  - GPU efficient
  - Stable learning
  - Faster than Batch GD

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 2 – Medical Image Diagnosis AI

- Scenario
  - Training MRI tumor detection CNN
  - Uses batches of images
- Why Mini-Batch?
  - Large dataset
  - Balanced speed + accuracy

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example 3 – Face Recognition Systems

- Scenario
  - Training on millions of face images
- Why Mini-Batch?
  - Memory efficient
  - Industry standard DL training

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Mini Batch GD

- Study **chapter-wise**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

When to use

Use Batch GD When

- Offline training
- Small dataset
- High precision required

Use SGD When

- Real-time learning
- Streaming data
- Personalization systems

Use Mini-Batch When

- Deep learning training
- GPU training
- Large datasets

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch GD vs SGD vs Mini Batch GD

Feature	Batch GD	SGD	Mini Batch
Data per Update	Entire dataset	One sample	Small batch
Speed	Slow	Fast	Medium-Fast
Stability	Very High	Low	Medium
Hardware Efficiency	Low	Medium	High
DL Usage	Rare	Rare	Very High

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch GD vs SGD vs Mini Batch GD

- **If Dataset = 1,000,000 samples**
  - Batch GD → 1 update per epoch
  - SGD → 1,000,000 updates per epoch
  - Mini Batch (Batch = 100) → 10,000 updates per epoch

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Numerical Example

- Minimize the function: $f(x) = x^2$ using Gradient Descent.
- Step 1: Identify Required Components
  - **Function** $f(x) = x^2$
  - **Gradient (Derivative)** $f'(x) = 2x$
  - **Learning Rate** Let: $\alpha = 0.1$
  - **Initial Value** Let: $x_0 = 5$
- Step 2: Gradient Descent Update Rule
  - $x_{new} = x_{old} - \alpha \cdot f'(x_{old})$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step-by-Step Iterations

- **Iteration 1**
  - **Current Value** $x = 5$
  - **Gradient** $f'(5) = 2(5) = 10$
  - **Update** $x = 5 - 0.1(10) = 5 - 1 = 4$
  - **New Function Value** $f(4) = 16$
- **Iteration 2**
  - **Current Value** $x = 4$
  - **Gradient** $f'(4) = 8$
  - **Update** $x = 4 - 0.1(8) = 4 - 0.8 = 3.2$
  - **New Function Value** $f(3.2) = 10.24$
- **Iteration 3**
  - **Current Value** $x = 3.2$
  - **Gradient** $f'(3.2) = 6.4$
  - **Update** $x = 3.2 - 0.1(6.4) = 3.2 - 0.64 = 2.56$
  - **New Function Value** $f(2.56) = 6.5536$
- **Iteration 4**
  - **Current Value** $x = 2.56$
  - **Gradient** $f'(2.56) = 5.12$
  - **Update** $x = 2.56 - 0.1(5.12) = 2.048$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Summary

Iteration	x Value	Gradient	f(x)
0	5	10	25
1	4	8	16
2	3.2	6.4	10.24
3	2.56	5.12	6.5536
4	2.048	4.096	4.19

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Gradient Descent

- What We Observe
  - x is moving toward **0**
  - Loss is decreasing each step
  - Gradient magnitude decreases near minimum
- **Why Convergence Happens**
  - At minimum: $f'(x) = 0$
  - For this function: Minimum at: $x = 0$
- **If Learning Rate is Too High?**
  - Example:
    If α = 1 → Overshooting happens
    If α = 0.0001 → Very slow learning

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

In Deep Learing

This Example	Deep Learning
x	Weight
f(x)	Loss
Gradient	Backpropagation gradient
Update	Weight update

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Adaptive Learning Rate Optimizers

- Adagrad
- RMSProp
- Adam

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Adagrad

Adagrad (Adaptive Gradient Algorithm) is a gradient-based optimization algorithm that **adapts the learning rate individually for each parameter** during training.

It is widely used in problems involving **sparse data** such as NLP and recommendation systems.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Core Idea of Adagrad

Instead of using fixed learning rate:

- Adagrad changes learning rate based on past gradients.

If parameter gets:

- Large gradients repeatedly → learning rate decreases
- Small gradients → learning rate remains higher

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Mathematical Formulation

- **Step 1: Gradient Calculation**

$$g_t = \nabla L(w_t)$$

- **Step 2: Accumulate Squared Gradients**

$$G_t = G_{t-1} + g_t^2$$

- **Step 3: Weight Update**

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} g_t$$

Symbol	Meaning
α	Learning rate
Gt	Sum of past squared gradients
ε	Small constant (avoid divide by zero)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Intuition

If parameter frequently receives gradient updates:

Learning rate decreases

Training stabilizes

If parameter rarely updated

Learning rate remains high

Faster learning for rare features

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Algorithm Steps

- Initialize weights
- Initialize gradient accumulator G = 0
- For each iteration:
  - Compute gradient
  - Update accumulator
  - Update weight

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Numerical Example

- Assume:
  - Weight w = 5
  - Learning rate α = 0.1
  - Initial G = 0
- **Iteration 1**
  - Gradient: g = 2w = 10
  - Accumulator: G = 100
  - Update:

$$w = 5 - \frac{0.1 \times 10}{\sqrt{100}} = 4.9$$

- **Iteration 2**
  - Gradient: g = 2(4.9) = 9.8
  - Accumulator: G = 100 + 96.04 = 196.04
  - Update step smaller → learning slows gradually.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Python Example

```
import numpy as np

w = 5
lr = 0.1
G = 0
eps = 1e-8

for i in range(20):
    grad = 2*w
    G += grad**2
    w = w - lr*grad/(np.sqrt(G)+eps)
    print("Step", i, "Weight:", w)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

TensorFlow Adagrad Example

```
import tensorflow as tf

model = tf.keras.Sequential([tf.keras.layers.Dense(1)])

optimizer = tf.keras.optimizers.Adagrad(learning_rate=0.01)

model.compile(optimizer=optimizer, loss='mse')
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

RMS prop (Root Mean Square Propagation)

RMSProp (Root Mean Square Propagation) is an adaptive gradient-based optimization algorithm used to improve training speed and stability in deep learning models.

It is especially useful when gradients change rapidly or when training sequence models.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why RMSProp Was Developed

Earlier optimizers had limitations

Problem in Standard Gradient Descent

- Same learning rate for all parameters
- May oscillate or converge slowly

Problem in Adagrad

- Learning rate keeps decreasing
- Eventually training stops early

RMSProp fixes this by using **moving average of squared gradients** instead of accumulating all gradients.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Core Idea of RMSProp

Instead of storing:
Total sum of squared gradients (Adagrad)

RMSProp stores:
Moving average of recent squared gradients

This prevents the learning rate from shrinking too fast.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Mathematical Formulation

- **Step 1: Compute Gradient**

$$g_t = \nabla L(w_t)$$

- **Step 2: Moving Average of Squared Gradients**

$$E\left[g^2\right]_t = \beta E\left[g^2\right]_{t-1} + (1-\beta) g_t^2$$

β ≈ 0.9 (decay factor)

- **Step 3: Weight Update**

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{E\left[g^2\right]_t + \epsilon}} g_t$$

α = learning rate and ε = small constant (avoid divide by zero)

where

$E\left[g^2\right]_t$ is the running average of squared gradients

$\beta \in [0, 1)$ is the decay factor (typically $\beta \approx 0.9$)

g_t^2 is the element-wise square of the gradient

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Intuition Behind RMSProp

If gradients are:

Large → Step size becomes smaller

Small → Step size becomes larger

Automatically balances learning rate per parameter.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

RMSProp Algorithm Steps

- Initialize weights
- Initialize moving average Eg = 0
- For each iteration:
  - Compute gradient
  - Update moving average
  - Update weights

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Numerical Example

- Assume: Weight w = 5, Learning rate α = 0.01, β = 0.9, Eg = 0
- **Iteration 1**
  - Gradient: g = 2w = 10
  - Moving Average: Eg = 0.9(0) + 0.1(100) = 10
  - Weight Update:

$$w = 5 - \frac{0.01 \times 10}{\sqrt{10}} \approx 4.968$$

- **Iteration 2**
  - Gradient decreases → smoother update → stable convergence.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Python Example – RMSProp

```
import numpy as np

w = 5
lr = 0.01
Eg = 0
beta = 0.9
eps = 1e-8

for i in range(20):
    grad = 2*w
    Eg = beta*Eg + (1-beta)*(grad**2)
    w = w - lr*grad/(np.sqrt(Eg)+eps)
    print("Step", i, "Weight:", w)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

TensorFlow RMSProp Example

```
import tensorflow as tf

model = tf.keras.Sequential([tf.keras.layers.Dense(1)])

optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001)

model.compile(optimizer=optimizer, loss='mse')
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-World Applications

Speech Recognition

Sequence learning with varying gradients.

Financial Time Series Prediction

Data patterns change frequently.

RNN / LSTM Training

Handles exploding / vanishing gradient issues better than basic GD.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

ADAM optimiser

Adam (Adaptive Moment Estimation) is one of the most widely used optimization algorithms in deep learning.

It combines the advantages of:

Momentum Optimization → Uses past gradients (1st moment)

RMSProp → Uses squared gradients (2nd moment)

Because of this combination, Adam provides **fast convergence + stable training**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why Adam Was Introduced

- Earlier optimizers had limitations:

Optimizer	Problem
Gradient Descent	Slow convergence
SGD	Noisy updates
Adagrad	Learning rate keeps decreasing
RMSProp	No momentum information

Adam solves all these by combining **momentum + adaptive learning rate**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Core Idea of Adam

- Adam maintains two values:
  - **First Moment (Mean of Gradients)**
    - Tracks average gradient → Like momentum → Smooth direction.
  - **Second Moment (Variance of Gradients)**
    - Tracks squared gradients → Adjusts learning rate → Stable updates.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Mathematical Formulation

- **Step 1: Gradient**

$$g_t = \nabla L(w_t)$$

- **Step 2: First Moment Estimate (Momentum)**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Typically: $\beta_1 = 0.9$

- **Step 3: Second Moment Estimate (RMSProp Idea)**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Typically: $\beta_2 = 0.999$

- **Step 4: Bias Correction**
- Since initial values are zero:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

- **Step 5: Final Weight Update**

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Intuition

If gradients:

Change direction → Momentum smooths

Change magnitude → Adaptive learning adjusts

Result → Fast + stable learning.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Adam Algorithm Steps

- Initialize parameters
- Initialize m = 0, v = 0
- For each iteration:
  - Compute gradient
  - Update m and v
  - Bias correction
  - Update weight

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Numerical Example

- Assume:
  - Weight $w = 5$
  - Learning rate $\alpha = 0.01$
  - $\beta 1 = 0.9$
  - $\beta 2 = 0.999$
- **Iteration 1**
  - Gradient: $g = 2w = 10$
  - First Moment: $m = 0.1 \times 10 = 1$
  - Second Moment: $v = 0.001 \times 100 = 0.1$
  - After correction → update weight slightly.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Python Example – Adam

```
import numpy as np

w = 5
lr = 0.01
m, v = 0, 0
b1, b2 = 0.9, 0.999
eps = 1e-8

for t in range(1,21):
    grad = 2*w

    m = b1*m + (1-b1)*grad
    v = b2*v + (1-b2)*(grad**2)

    m_hat = m/(1-b1**t)
    v_hat = v/(1-b2**t)

    w = w - lr*m_hat/(np.sqrt(v_hat)+eps)
    print("Step", t, "Weight:", w)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

TensorFlow Adam Example

```
import tensorflow as tf

model = tf.keras.Sequential([ tf.keras.layers.Dense(1) ])

optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

model.compile(optimizer=optimizer, loss='mse')
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-World Applications

Computer Vision

- CNN image classification training.

NLP Models

- Transformers, language models.

Speech Recognition

- Sequence deep learning models.

Recommendation Systems

- Large-scale user behavior data.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

When to Use Adam

- Default starting optimizer
- Deep neural networks
- Noisy gradient problems
- Large dataset training

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Unit Saturation

Unit Saturation is a problem in neural networks where neurons become stuck in regions where the gradient is almost zero, causing **very slow learning or no learning**.

It mainly occurs in activation functions like **Sigmoid** and **Tanh**.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is Unit Saturation?

Unit Saturation happens when:

- Neuron output becomes extremely high or extremely low
- Activation function output becomes flat
- Gradient becomes very small (≈ 0)
- Weight updates stop or become very slow

This leads to **vanishing gradient problem**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Where Unit Saturation Occurs

- **Sigmoid Function**
  - Output Range: $0 \; to \; 1$
  - When input is:
    - Very large positive → Output ≈ 1
    - Very large negative → Output ≈ 0
  - Gradient ≈ 0 → Learning stops.
- **Tanh Function**
  - Output Range: $-1 \; to \; 1$
  - When input is large → Gradient becomes very small.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why Unit Saturation is Dangerous

Vanishing Gradient

Backpropagation depends on gradient.

If gradient ≈ 0 → No weight update.

Slow Training

Network learns extremely slowly.

Deep Network Failure

Early layers stop learning completely.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Mathematical Understanding

- For Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Derivative: $\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big)$
- When σ(x) ≈ 0 or 1: Derivative ≈ 0 → Saturation.

- Practical Example
  - If neuron input = 10, Sigmoid(10) ≈ 0.99995
  - Derivative ≈ 0 → Very small gradient → Very slow learning.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How Modern Deep Learning Solves Unit Saturation

- **ReLU Activation**
  - Output: $\max(0, x)$
  - Does not saturate for positive values.
- **Leaky ReLU**
  - Allows small gradient even for negative inputs.
- **Batch Normalization**
  - Keeps neuron inputs in safe range.
- **Proper Weight Initialization**
  - Prevents very large activations.
- **Residual Networks (ResNet)**
  - Helps gradient flow.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Python Demonstration (Saturation Effect)

```
import numpy as np

def sigmoid(x):
    return 1/(1+np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s*(1-s)

values = [-10, -5, 0, 5, 10]

for v in values:
    print("Input:", v,
          "Sigmoid:", sigmoid(v),
          "Derivative:", sigmoid_derivative(v))
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Simple Real-Life Analogy

Neuron = Student

If student is:

Too confident → Stops learning

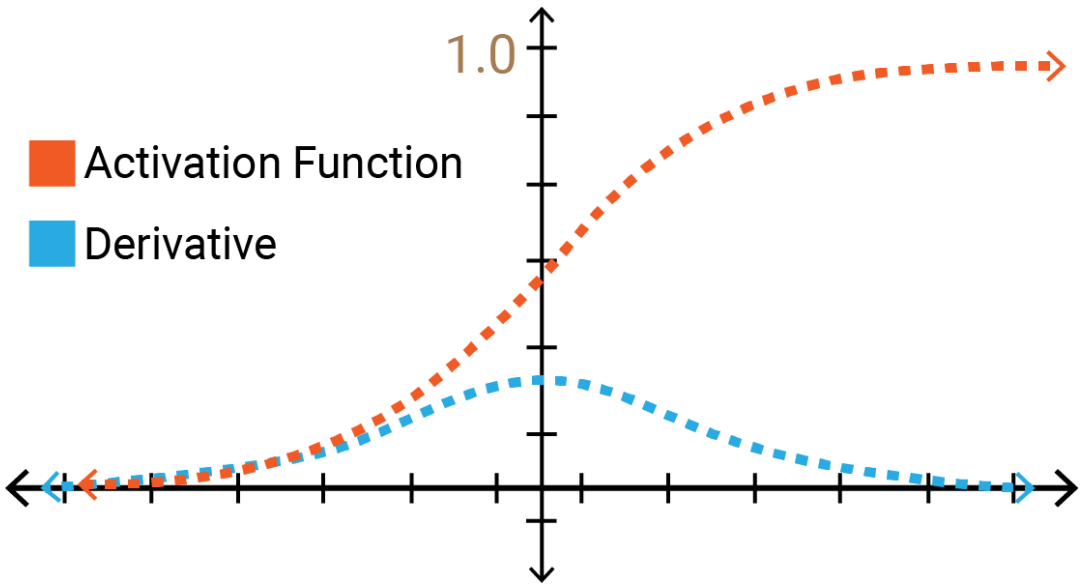
Too confused → Cannot learn

Balanced state → Best learning.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

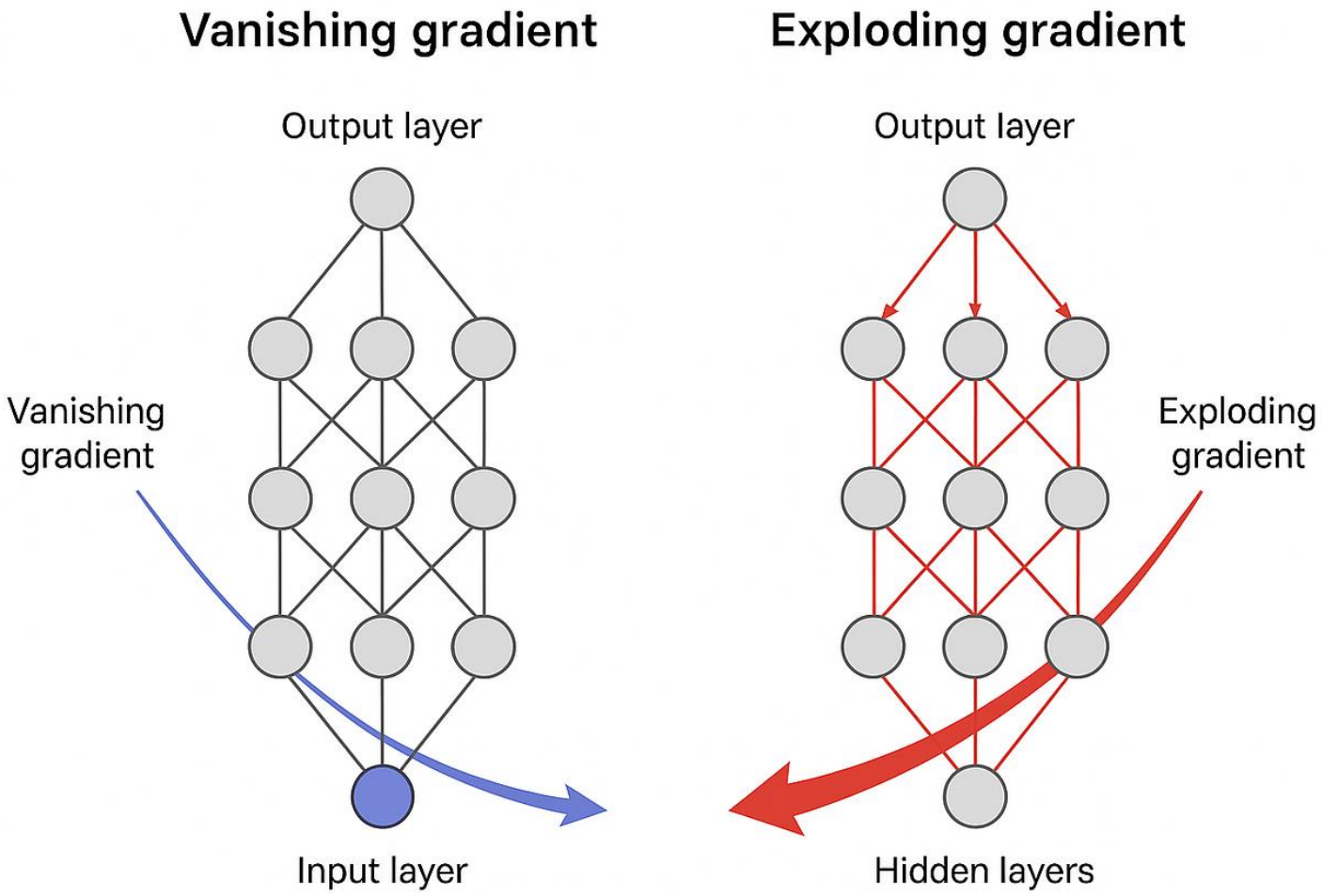
Vanishing and Exploding Gradient

- Vanishing and Exploding Gradients are major problems encountered while training **deep neural networks**, especially during **backpropagation**.
- These problems affect how gradients flow from output layer back to earlier layers.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Vanishing and Exploding Gradient



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Vanishing Gradient Problem

- Gradients become **very small (close to zero)** as they move backward through layers.
- **Why It Happens**
  - Occurs due to:
    - Sigmoid activation
    - Tanh activation
    - Deep networks (many layers)
  - Because derivatives are < 1 → multiplying many times → becomes ≈ 0.
- Mathematical Intuition
  - If gradient multiplier per layer = 0.5
  - After 10 layers: $0.5^{10} = 0.00097$
  - Gradient almost zero.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Vanishing Gradient Problem

Effects

- Early layers stop learning
- Training becomes very slow
- Poor feature extraction

Real Example

- Old RNN models → Cannot learn long sequences.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Exploding Gradient Problem

- Gradients become **very large** during backpropagation.
- **Why It Happens**
  - Occurs due to:
    - Large weight initialization
    - Deep networks
    - Repeated multiplication of gradients > 1
- **Mathematical Intuition**
  - If gradient multiplier per layer = 2
  - After 10 layers: $2^{10} = 1024$
  - Gradient becomes huge → unstable updates.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Exploding Gradient Problem

Effects

Training becomes unstable

Loss oscillates or becomes NaN

Weight values explode

Real Example

Deep RNNs before gradient clipping → unstable training.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Vanishing vs Exploding Gradient

Feature	Vanishing Gradient	Exploding Gradient
Gradient Value	Very Small	Very Large
Learning Speed	Very Slow	Unstable
Weight Updates	Almost Zero	Too Large
Common In	Sigmoid / Tanh Networks	Poor Initialization / Deep RNN

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Solutions to Vanishing Gradient

ReLU Activation	• Does not saturate for positive values.
Batch Normalization	• Keeps activations in stable range.
Proper Weight Initialization	• Xavier Initialization • He Initialization
Residual Networks (ResNet)	• Allows gradient shortcut flow.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Solutions to Exploding Gradient

Gradient Clipping	• Limit gradient magnitude.
Proper Initialization	• Avoid large weights.
LSTM / GRU Networks	• Control gradient flow in sequence models.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Simple Python Demonstration

- Vanishing Gradient

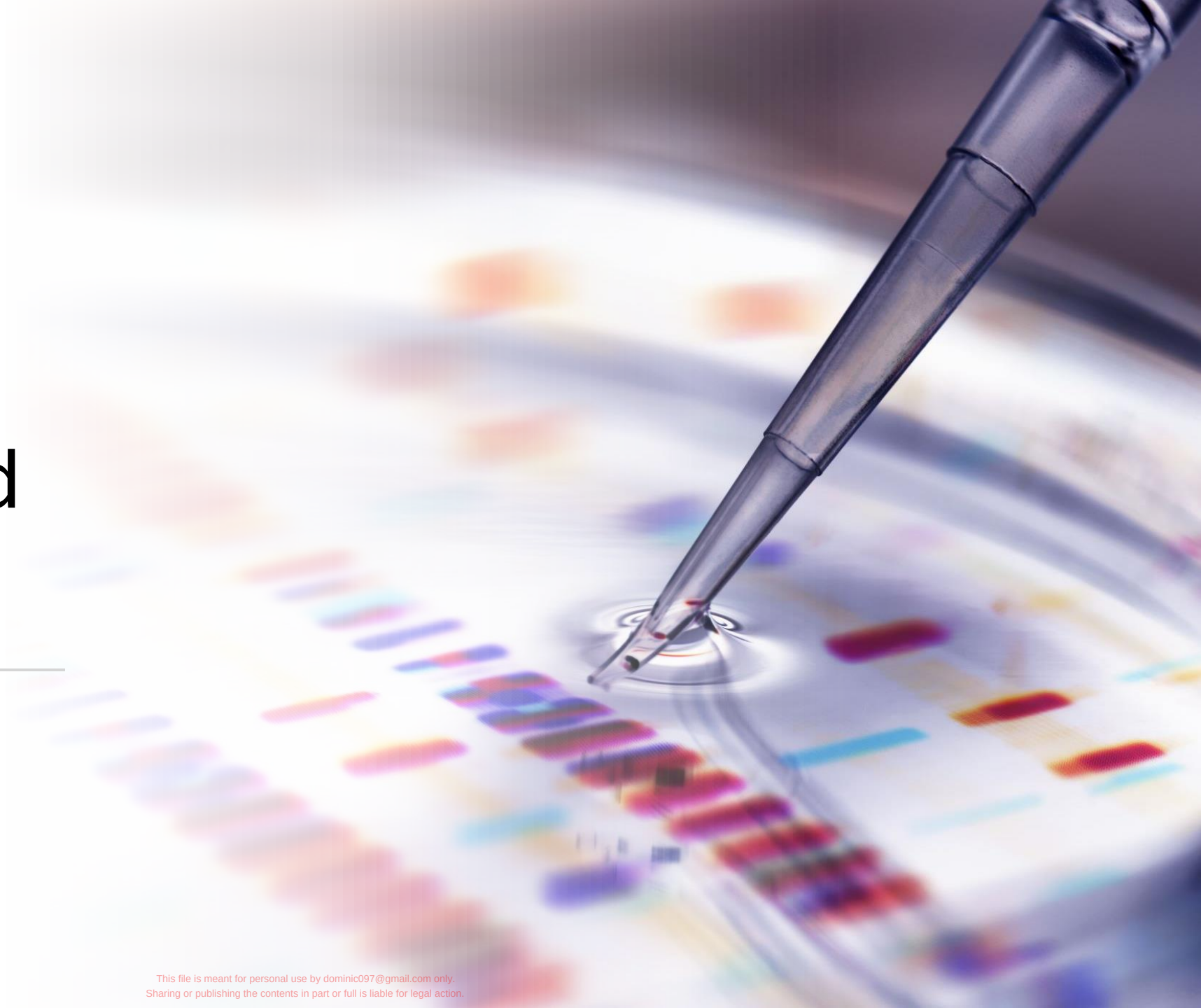
```
grad = 1
for i in range(10):
    grad *= 0.5
    print(grad)
```

- Exploding Gradient

```
grad = 1
for i in range(10):
    grad *= 2
    print(grad)
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Underfitting, Overfitting and Learning Rate



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is Underfitting?

- Underfitting occurs when model is too simple to learn patterns.
- Model cannot capture input-output relationship.
- **Underfitting Characteristics**
  - High training error
  - High testing error
  - Low accuracy
  - Poor pattern learning
- **Causes of Underfitting**
  - Simple model
  - Less training time
  - Missing important features
  - Too much regularization
- Real Example of Underfitting
  - Predict house price using only area → ignoring location, facilities.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How to Fix Underfitting

1. Increase model complexity
2. Train longer
3. Add features
4. Reduce regularization

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-Time Scenario – Underfitting

- **Medical Diagnosis AI**
  - **Situation**
    - Hospital builds AI model to detect tumors from MRI images.
  - **Mistake**
    - Uses very small neural network
    - Only 1 hidden layer
    - Trained for few epochs
  - **Real Outcome**
    - Misses tumors
    - High false negatives
    - Unsafe for patient use

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-Time Scenario – Underfitting

- **Speech Recognition App**
  - **Mistake**
    - Small dataset
    - Simple model architecture
  - **Result**
    - Cannot understand accents
    - Poor real-world performance
- **Self Driving Car Object Detection**
  - **Mistake**
    - Uses simple feature-based ML model
    - Instead of deep CNN
  - **Result**
    - Cannot detect complex road scenes

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is Overfitting?

- Overfitting occurs when model memorizes training data including noise.
- **Overfitting Characteristics**
  - Very low training error
  - High validation/test error
  - Poor real-world performance
- **Causes of Overfitting**
  - Complex model
  - Small dataset
  - Long training
  - Too many parameters

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is Overfitting?

- Real Example of Overfitting
  - Medical AI trained on small dataset → memorizes training images.
- **How to Fix Overfitting**
  - More data
  - Data augmentation
  - Dropout
  - Early stopping
  - Regularization

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-Time Scenario – Overfitting

- **Cancer Detection AI**
  - **Situation**
    - AI trained on only 200 patient images.
  - **Mistake**
    - Very deep CNN
    - No data augmentation
    - No dropout
  - **Real Outcome**
    - 99% training accuracy
    - 60% real patient accuracy

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-Time Scenario – Overfitting

- **Recommendation System**
  - **Mistake**
    - Model trained on last month user behavior only.
  - **Result**
    - Cannot generalize to new user trends
- **Face Recognition Security System**
  - **Mistake**
    - Trained using only office employees dataset.
  - **Result**
    - Cannot recognize visitors
    - High false rejection

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is Learning Rate?

Learning rate controls weight update step size during training.

Learning Rate Formula

New Weight = Old Weight – Learning Rate × Gradient

Learning Rate Too Small

Very slow convergence
High training time

Learning Rate Too Large

Overshoot minimum
Loss oscillation
Training instability

Optimal Learning Rate

Fast convergence
Stable training
Better accuracy

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Real-Time Scenario – Learning Rate Issues

- **Mobile Keyboard Prediction Model**
  - **Learning Rate Too Small**
  - Result:
    - Training takes weeks
    - Slow improvement
- **Autonomous Driving Model Training**
  - **Learning Rate Too Large**
  - Result:
    - Loss oscillates
    - Model never converges
    - Training crashes
- **Medical Image Deep Learning Training**
  - **Correct Learning Rate**
  - Result:
    - Stable training
    - Faster convergence
    - High accuracy

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Simple Real-Life Analogy

- **Underfitting**
  - Student studies only chapter summary → fails exam
- **Overfitting**
  - Student memorizes previous question paper → fails new paper
- **Learning Rate Wrong**
  - Student studies too fast → forgets
  - Student studies too slow → cannot finish syllabus

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Session-04

4.1 Regularization Techniques - LASSO Regression, Ridge Regression, Dropouts, Early Stopping

4.2 Normalization techniques in Neural Networks, Batch Normalization, Group Normalization, Instance Normalization

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

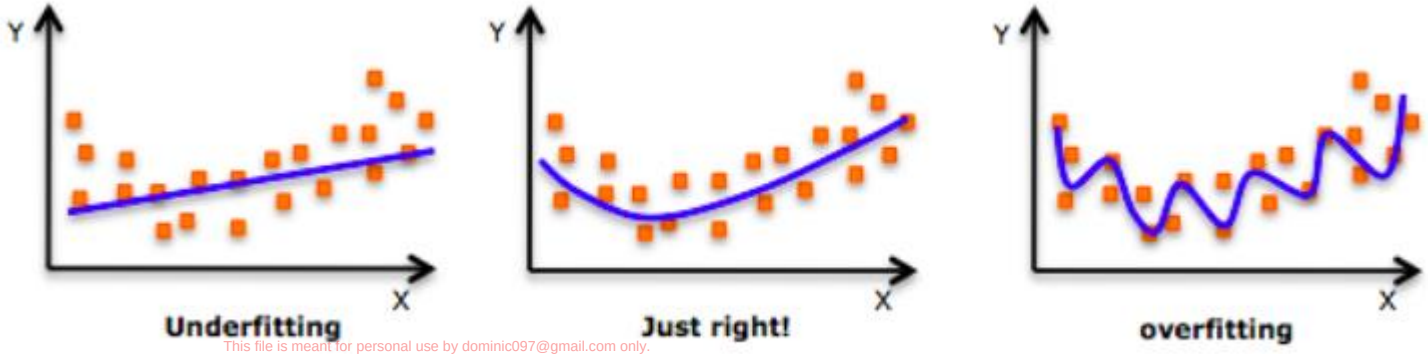
Regularization Techniques In Neural Networks

- Regularization techniques are methods used to **prevent overfitting** and improve the **generalization ability** of neural networks.
- They help the model learn true patterns rather than memorizing training data noise.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Correct fit vs. overfit models

- Correct fit model
  - If the model is able to perform well on the testing dataset, the model can be said to have generalized well,
  - i.e., correctly understood the patterns provided in the training dataset. This type of model is called a
- An overfit model
  - if the model performs really well on the training data and doesn't perform well on the testing data,
  - So, the model has memorized the patterns of training data but is not able to generalize well on unseen data.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Methods used to handle overfitting

- Training on more training data to better identify the patterns.
- Data augmentation for better model generalization.
- Early stopping, i.e., stop the model training when the validation metrics start decreasing or loss starts increasing.
- Regularization techniques.

- Data augmentation and more training data don't change the model architecture but try to improve the performance by altering the input data.
- Early stopping is used to stop the model training at an appropriate time - before the model overfits, rather than addressing the issue of overfitting directly.
- **However, regularization is a more robust technique that can be used to avoid overfitting.**

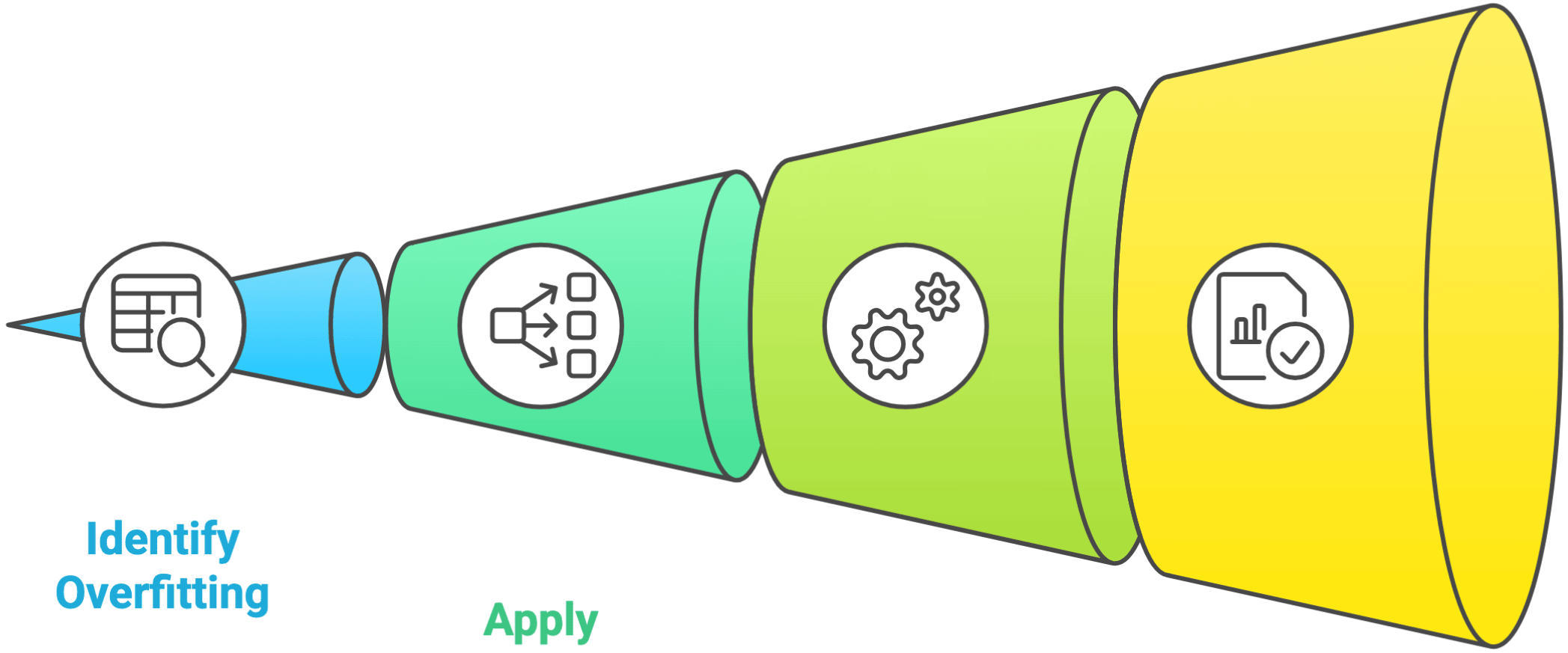
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Regularization

- Regularization is a technique used to address overfitting by directly changing the architecture of the model by **modifying the model's training process.**

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Regularization Process in Deep Learning



Identify Overfitting

Recognize high training accuracy but low test accuracy

Apply Regularization

Implement

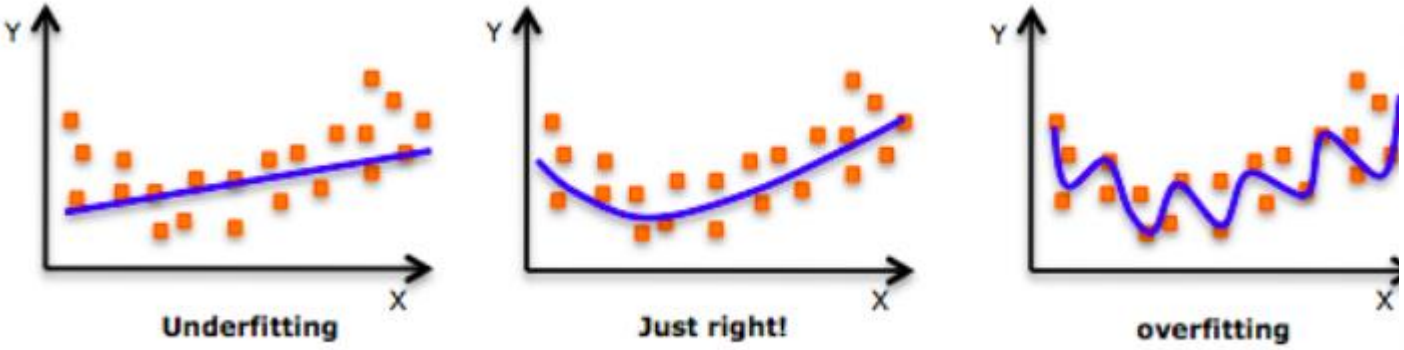
Adjust Model Complexity

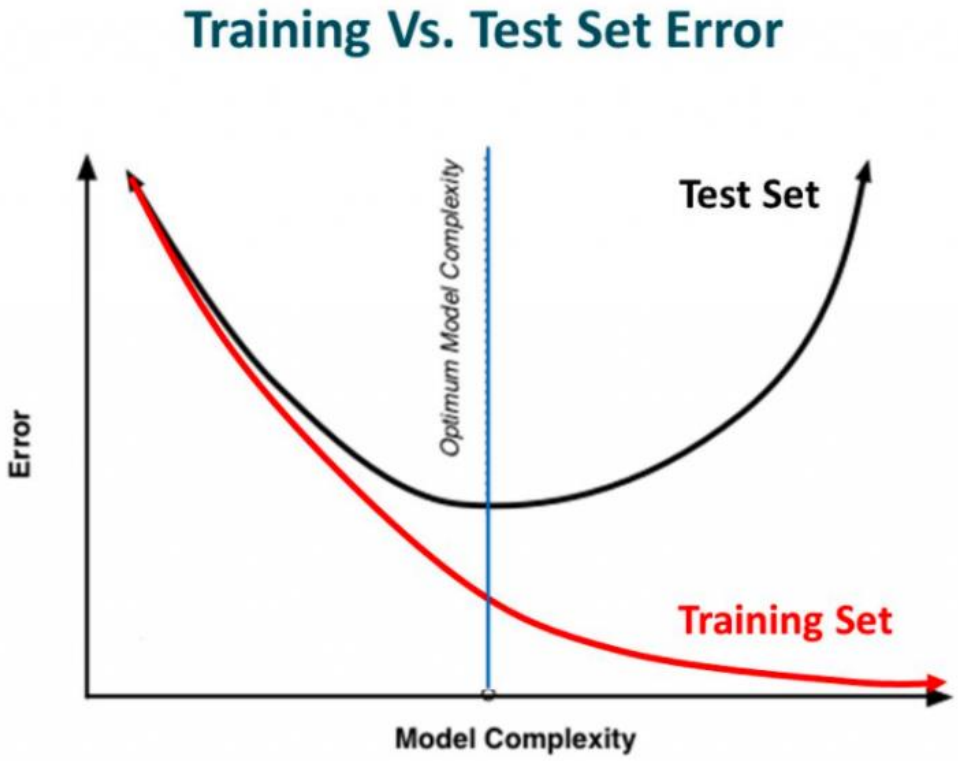
Evaluate

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

What is Regularization?

- while going towards the right, the complexity of the model increases such that the training error reduces but the testing error doesn't.



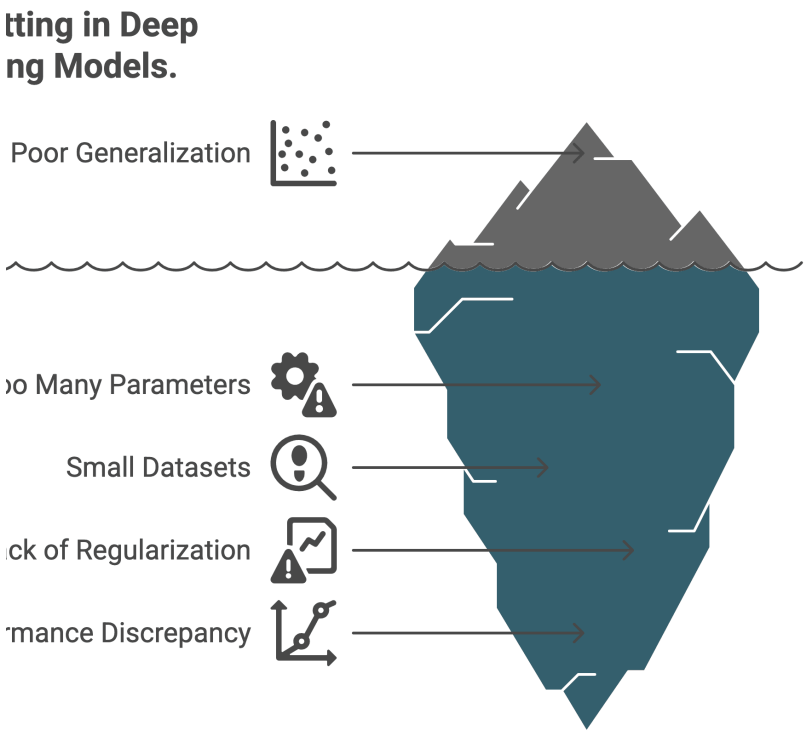


Regularization is a technique which makes slight modifications to the learning algorithm such that the model generalizes better.
This in turn improves the model's performance on the unseen data as well.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why Overfitting Happens in Deep Learning Models

- **Too Many Parameters**
  - A deep CNN trained on only 1,000 medical X-rays may achieve 99% training accuracy but perform poorly on unseen scans due to overfitting to patient-specific artifacts
- **Small Datasets**
  - In natural language processing, fine-tuning BERT on a sentiment analysis task with only 500 examples may cause the model to memorize specific word combinations without learning generalized sentiment patterns.
- **Lack of Noise or Dropout**
  - An image classifier trained without dropout or image augmentation (like rotation, flip, or crop) will overfit to specific lighting or background conditions.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Indicators of Overfitting

Training accuracy is high, while validation accuracy remains low or plateaus.

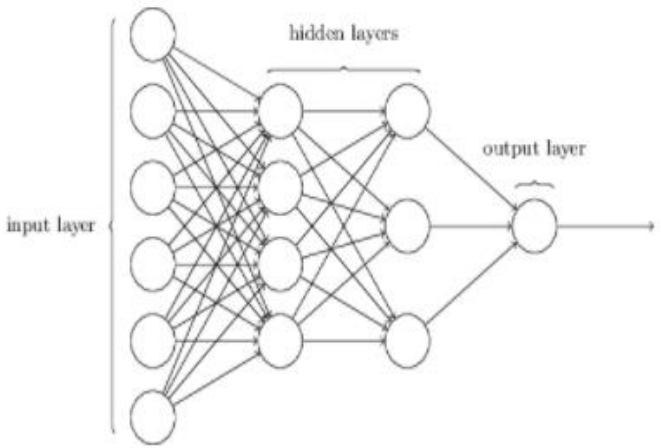
The gap between training and validation loss widens as training continues.

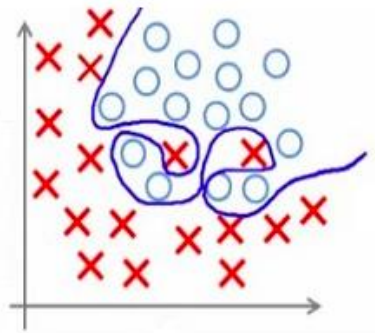
Model starts memorizing noise or rare patterns, leading to poor generalization.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How does Regularization help reduce Overfitting?

- Let's consider a neural network which is overfitting on the training data.
- In machine learning, regularization penalizes the coefficients.
- In deep learning, it actually penalizes the weight matrices of the nodes.





Over-fitting

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Why Regularization is Needed

Problem Without Regularization

Neural networks may memorize training data

Poor performance on unseen data

High variance models

Goal of Regularization

Reduce overfitting

Improve generalization

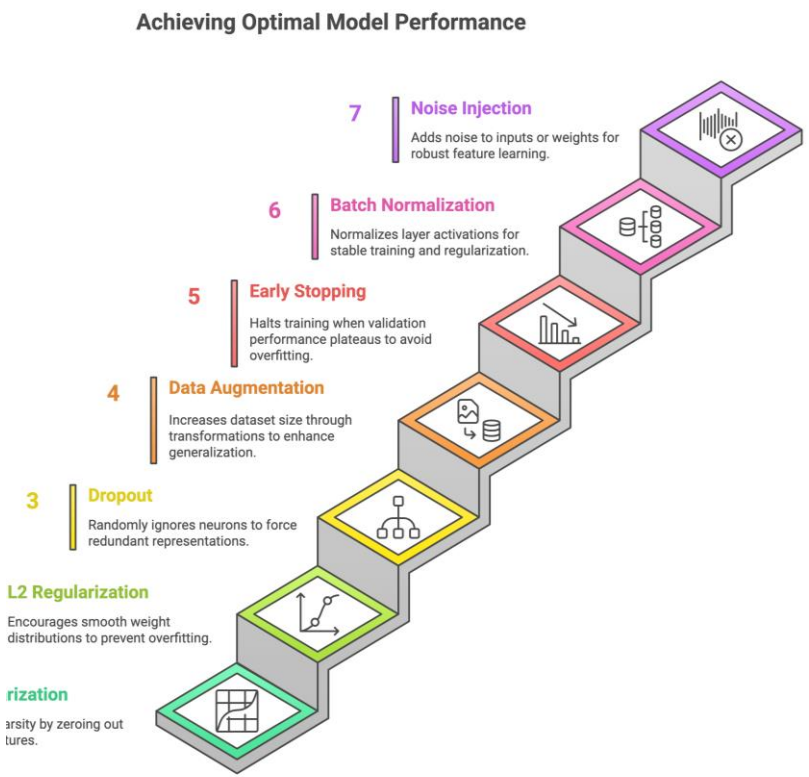
Stabilize training

Improve model robustness

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Regularization Techniques

- L1 Regularization (Lasso Regularization)
- L2 Regularization (Ridge Regularization)
- Dropout Regularization
- Early Stopping
- Data Augmentation



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L1 Regularization (Lasso Regularization)

- The absolute value of the magnitude of coefficients or weights multiplied with a regularizer term is added to the loss or cost function

$$\underbrace{\sum_{i=1}^{n}\left(y_i - \sum_{j=1}^{p} x_{ij} b_j\right)^2}_{Loss\ Term} + \underbrace{\lambda \cdot \sum_{j=1}^{p} |b_j|}_{Regularizer\ term}$$

- A fraction of the sum of absolute values of weights to the loss function is added in the L1 regularization
- In this way, you will be able to eliminate some coefficients with lesser values by pushing those values towards 0

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L1 Regularization (Lasso Regularization)

- Observe the following by using L1 regularization:
  - Since the L1 regularization adds an absolute value as a penalty to the cost function, the feature selection will be done by retaining only some important features and eliminating the lower or unimportant features.
  - This technique is also robust to outliers, i.e., the model will be able to easily learn about outliers in the dataset.
  - This technique will not be able to learn complex patterns from the input data.
- **Use Case**
  - In high-dimensional datasets, such as those in gene expression analysis, L1 can aid in selecting a small subset of relevant genes.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Example

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression

# Create a synthetic dataset for demonstration
X, y = make_regression(n_samples=100, n_features=10, noise=0.1, random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Apply Lasso Regression (L1 Regularization)
lasso = Lasso(alpha=0.1)  # alpha is equivalent to λ (regularization strength)
lasso.fit(X_train, y_train)

# Predict and evaluate the model
predictions = lasso.predict(X_test)

# Output results
print("Model coefficients:", lasso.coef_)
print("Training score:", lasso.score(X_train, y_train))
print("Testing score:", lasso.score(X_test, y_test))
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L1 Regularization (Lasso Regularization)

- LASSO adds penalty equal to **absolute value of weights**.
- Helps in **feature selection**
- Some weights become exactly zero
- **Mathematical Formula**

$$Loss = Original\ Loss + \lambda \sum |w|$$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L1 Regularization (Lasso Regularization)

- Assume:
  - Model prediction error (MSE) = 10
  - Weights: $w_1 = 2, w_2 = -1, w_3 = 0.5$
  - $\lambda = 0.5$
- L1 penalty

$$| 2 |+| -1 |+| 0.5 |= 3.5$$

- Regularized loss

$$\text{Cost Function}, J = Loss + \lambda \sum | w_i |$$

$$J = 10 + 0.5 \times 3.5 = 11.75$$

- During training, small weights shrink to zero
  Example after updates: $w_3 \rightarrow 0$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question

- A linear regression model for predicting house prices has the following weight values after training:

$$w = \text{w=[3,−2,1,0.5]}$$

The mean squared error (MSE) is **12**, and the L1 regularization parameter $\lambda = 0.4$.

- **Tasks:**
  - Compute the L1 regularization penalty.
  - Calculate the total cost function.
  - Identify which feature is most likely to be eliminated after regularization and justify.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question 2

- A dataset contains **6 features**, but only **3 features are relevant**. The trained model has the following weights:

$$w = \text{w=[2.5,0.1,−0.05,1.8,0.02,−0.01]}$$

- Given $\lambda = 0.6$:
  - Compute the L1 penalty.
  - Show how L1 regularization performs feature selection.
  - Explain why Lasso is suitable for this scenario.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L2 Regularization (Ridge Regularization)

- the squared magnitude of the coefficients or weights multiplied with a regularizer term is added to the loss or cost function.
- Mathematical equation:

$$\underbrace{\sum_{i=1}^{n}\left(y_i - \sum_{j=1}^{p} x_{ij} b_j\right)^2}_{Loss\ term} + \underbrace{\lambda \cdot \left(\sum_{j=1}^{p} b_j\right)^2}_{Regularizer\ term}$$

y_i – *labels*
x_{ij} – *features*
b_j – *weights*
λ – *regularizer term (lambda)*
i – *no of training samples*

- A fraction of the sum of squared values of weights is added to the loss function.
- Thus, when gradient descent is applied on loss, the weight update seems to be consistent by giving almost equal emphasis on all features.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L2 Regularization (Ridge Regularization)

- Observe the following:
  - Lambda is the hyperparameter that is tuned to prevent overfitting i.e. penalize the insignificant weights by forcing them to be small but not zero.
  - L2 regularization works best when all the weights are roughly of the same size, i.e., input features are of the same range.
  - This technique also helps the model to learn more complex patterns from data without overfitting easily.
- **Use Case**
  - In time-series forecasting, L2 regularization helps smooth out fluctuating weights, which is useful when dealing with noisy data, enabling the model to capture trends without overfitting or anomaly detection issues.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L2 Regularization (Ridge Regularization)

- Adds penalty equal to **square of weights**.
- Reduces weight magnitude
- Does NOT remove features completely
- **Mathematical Formula**

$$Loss\ (\mathrm{J}) = Original\ Loss + \lambda \sum w^2$$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

L2 Regularization (Ridge Regularization)

- Assueme:
  - Loss = 10
  - Weights: $2, -1, 0.5$
  - $\lambda = 0.5$
- L2 penalty

$$2^2 + (-1)^2 + 0.5^2 = 5.25$$

- Regularized loss

$$J = 10 + 0.5 \times 5.25 = 12.625$$

- Weights after training:

$$w_1 = 1.5,\ w_2 = -0.7,\ w_3 = 0.2$$

- ✓All features retained, but less dominant

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question

- A ridge regression model has weights:
$$w = [2, -1, 0.5]$$
- Training loss = **8**, regularization parameter $\lambda = 0.3$.
- **Tasks:**
  - Calculate the L2 regularization penalty.
  - Compute the total loss.
  - Explain how L2 regularization affects large weights numerically.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question

- In a medical diagnosis system, the trained model produces large weight values:

$$w = [4.5, -3.2, 2.8, -1.5]$$

- Given $\lambda = 0.5$:
  - Compute the L2 penalty.
  - Calculate the updated loss if original loss = **15**.
  - Analyze how Ridge regularization improves model stability.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

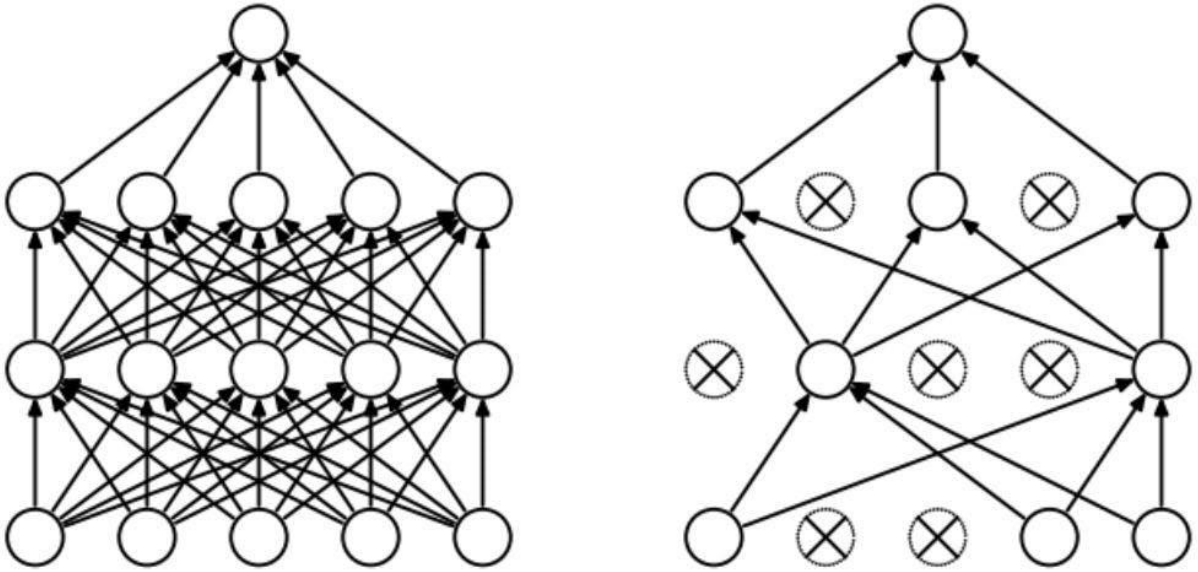
Example

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression

# Create a synthetic dataset for demonstration
X, y = make_regression(n_samples=100, n_features=10, noise=0.1,
random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Apply Ridge Regression (L2 Regularization)
ridge = Ridge(alpha=0.1)  # alpha is equivalent to λ
(regularization strength)
ridge.fit(X_train, y_train)

# Predict and evaluate the model
predictions = ridge.predict(X_test)

# Output results
print("Model coefficients:", ridge.coef_)
print("Training score:", ridge.score(X_train, y_train))
print("Testing score:", ridge.score(X_test, y_test))
```

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Dropout regularization

- In this method, some of the neurons are randomly disabled during the training such that the model can extract more useful robust features from the model.
- This prevents overfitting.



(a) Standard Neural Net (b) After applying dropout.

In figure (a), the neural network is fully connected. If all the neurons are trained with the entire training dataset, some neurons might memorize the patterns occurring in training data. This leads to overfitting since the model is not generalizing well.

In figure (b), the neural network is sparsely connected, i.e., only some neurons are active during the model training. This forces the neurons to extract robust features/patterns from training data to prevent overfitting.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Dropout Regularization Characteristics

- Dropout randomly disables some percent of neurons in each layer. So for every epoch, different neurons will be dropped leading to effective learning.
- Dropout is applied by specifying the ‘p’ values, which is the fraction of neurons to be dropped.
- Dropout reduces the dependencies of neurons on other neurons, resulting in more robust model behavior.
- Dropout is applied only during the model training phase and is not applied during the inference phase.
- When the model receives complete data during the inference time, you need to scale the layer outputs ‘x’ by ‘p’ such that only some parts of data will be sent to the next layer. This is because the layers have seen less amount of data as specified by dropout.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Dropout Regularization

- Randomly disables neurons during training.
- Prevents neuron co-dependency
- Improves generalization

Use Case

In deep neural networks, such as CNNs and RNNs, dropout is often employed to prevent overfitting when the model is large and has many layers.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How to Fix (Choose) the Dropout Rate

- Dropout rate is a **hyperparameter** chosen based on **layer type, data size, and overfitting level**.
- 1. Layer Type

Layer Type	Typical Dropout Rate
Input layer	**0.1 – 0.2**
Hidden layers (Dense)	**0.3 – 0.5**
Deep networks	**0.5**
CNNs	**0.2 – 0.4**
RNN / LSTM	**0.1 – 0.3**

  - Start with **0.5 for dense layers**
  - Reduce if underfitting occurs

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How to Fix (Choose) the Dropout Rate

- 2. Fix Dropout Using Validation Loss (Best Method)
  - **Procedure**
  - Choose candidate rates

$$\{0.1, \ , 0.3, \ 0.5 \ 0.7\}$$

  - Train model
  - Observe **validation loss**

Dropout Rate	Training Loss	Validation Loss
0.1	0.20	**0.45**
0.3	0.25	**0.32**
0.5	0.35	**0.28** ✅
0.7	0.60	0.50

  - **Best choice = 0.5** (lowest validation loss)

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How to Fix (Choose) the Dropout Rate

- 3. Based on Overfitting Behavior
  - **Case 1: Overfitting**
    - Training loss ↓
    - Validation loss ↑
    - **Increase dropout** $0.3 \rightarrow 0.5$
  - **Case 2: Underfitting**
    - Both losses high
    - **Decrease dropout** $0.5 \rightarrow 0.2$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How to Fix (Choose) the Dropout Rate

- Layer-Wise Dropout Fixing (Recommended)

Layer	Dropout
Input	0.1
Hidden-1	0.3
Hidden-2	0.5
Output	❌ No dropout

  - Preserves information early
  - Strong regularization deeper

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Dropout Regularization

- Hidden layer activations:

$$[1.0, 0.8, 0.4, 0.2]$$

- Dropout rate = 0.5
- Random mask:

$$[1, 0, 1, 0]$$

- After dropout:

$$[1.0, 0, 0.4, 0]$$

- Some neurons do not participate in this iteration
- During testing:
  - No *dropout, outputs scaled*

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question

- A hidden layer contains **10 neurons**. A dropout rate of **0.4** is applied.
- **Tasks:**
  - Calculate the expected number of active neurons during training.
  - If neuron activations are [1.2,0.8,0.5,0.3,1.0,0.9,0.6,0.4,0.2,0.7], apply a dropout mask [1,0,1,0,1,1,0,1,0,1] and compute the output.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question

- A neural network trained for image classification shows overfitting. Two dropout rates are tested:

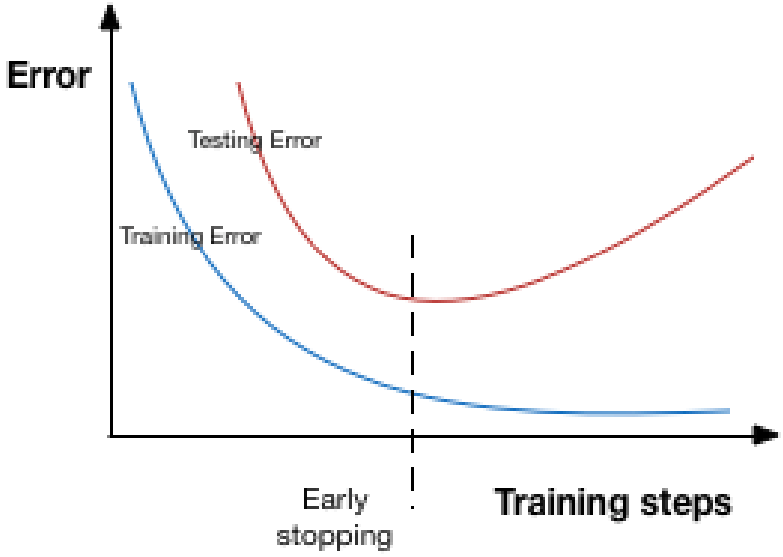
Dropout Rate	Training Loss	Validation Loss
0.2	0.25	0.48
0.5	0.38	0.31

- **Tasks:**
  - Identify the optimal dropout rate.
  - Explain numerically why the selected dropout rate improves generalization.
  - Predict the effect of increasing dropout to 0.7.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Early Stopping

- Early stopping involves monitoring the validation loss during training and halting the process once the validation performance stops improving.
- This prevents the model from overfitting by stopping training before it begins to memorize noise in the training data.



Use Case: In tasks like time-series forecasting, where excessive training can cause models to fit to noise, early stopping ensures optimal performance without overfitting.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Early Stopping Example

- Stop training when **validation loss increases**
- Prevents overfitting even if training loss decreases

Epoch	Training Loss	Validation Loss
1	1.2	1.3
2	0.9	1.0
3	0.6	0.8
4	0.4	**0.9** ↑
5	0.3	1.1

- Stop at **Epoch 3**
- Best generalization achieved before overfitting starts

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question

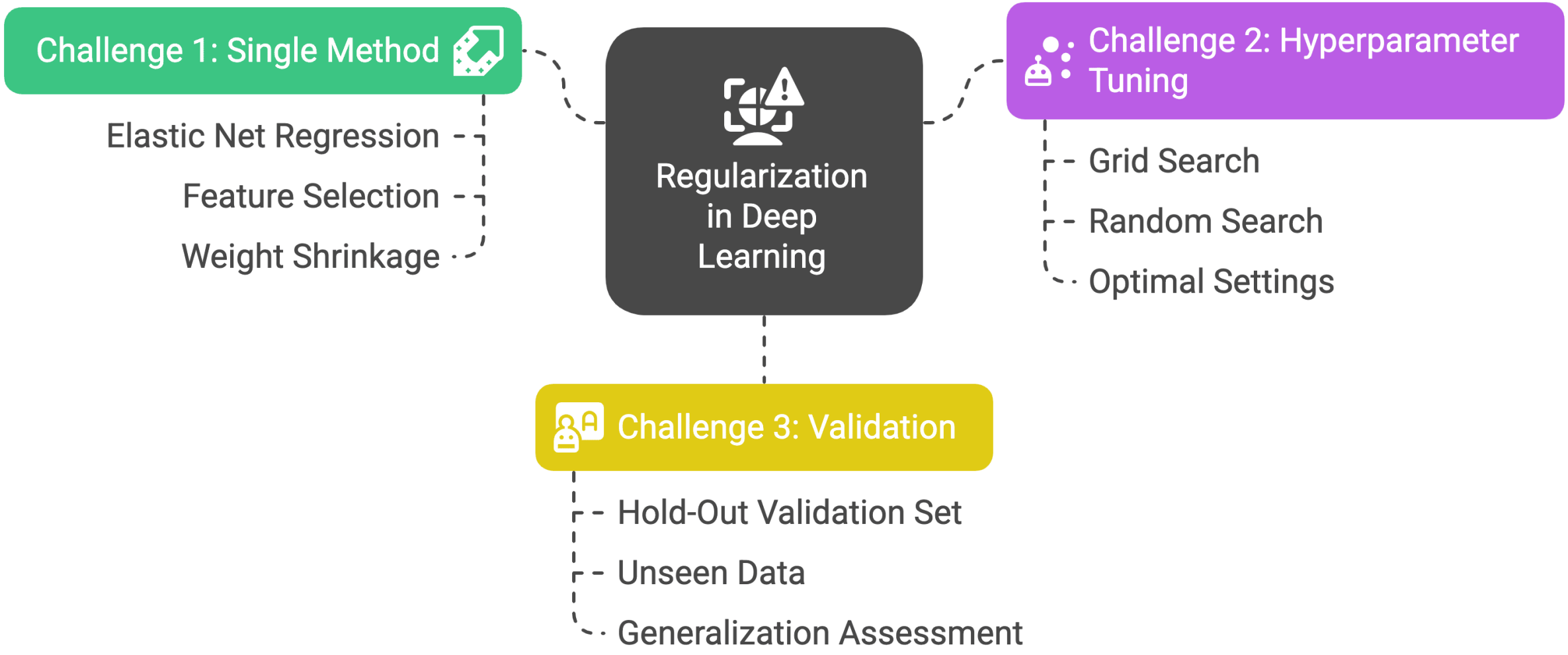
- During training of a deep learning model, the following validation losses are observed:

Epoch	Validation Loss
1	0.95
2	0.72
3	0.60
4	0.62
5	0.70

- **Tasks:**
  - Identify the epoch at which early stopping should be applied.
  - Justify your answer numerically.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Best Practices for Applying Regularization in Deep Learning



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

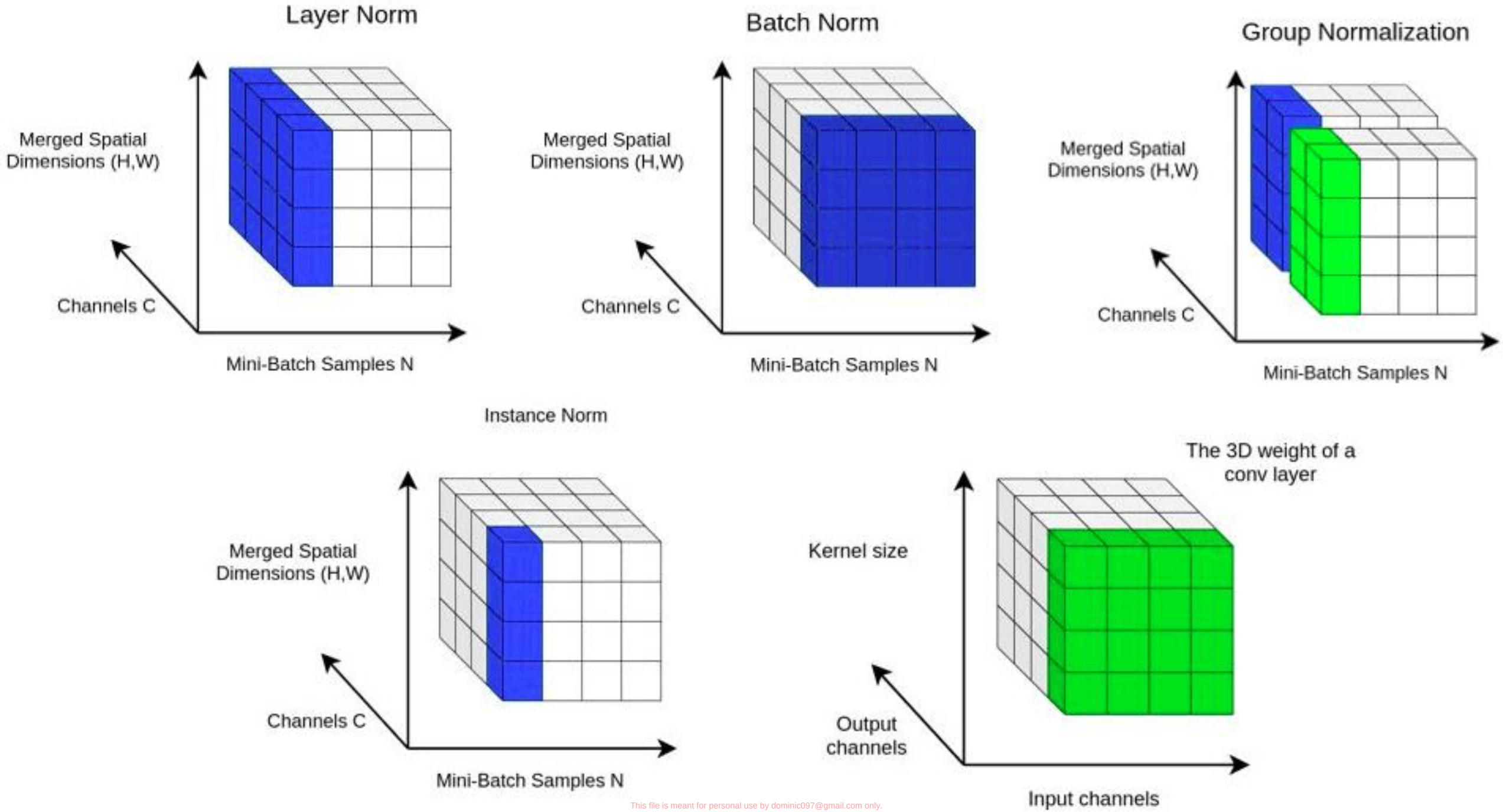
Normalization techniques in Neural Networks

Batch Normalization

Group Normalization

Instance Normalization

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch Normalization

- **Definition**
  - **Batch Normalization** is a technique that normalizes neuron activations using the **mean and variance of a mini-batch** during training, followed by learnable scale and shift parameters.
- **Objective**
  - Reduce **internal covariate shift**
  - Stabilize and accelerate training
  - Allow higher learning rates
- **Working Principle**
  - Compute **mean and variance** of activations over a mini-batch
  - Normalize activations to zero mean and unit variance
  - Apply learnable **scale (γ)** and **shift (β)** parameters
  - During inference, use **running averages** instead of batch statistics
- **Applications**
  - CNNs (ImageNet models)
  - Deep feedforward networks
  - Large-batch training

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch Normalization

- **Step 1: Compute Batch Mean**

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

- m: batch size
- x_i :activation value of neuron for sample i
- This computes the average activation over the batch

- *Why?* → To know the center of data.

- **Step 2: Compute Batch Variance**

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_B)^2$$

- Measures **spread** of activations
- Tells how far values deviate from the mean

- *Why?* → To control **scale differences** across neurons.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch Normalization

- **Step 3: Normalize the Activations**

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

  - $x_i - \mu_B$ →centers data at zero
  - $\sqrt{\sigma_B^2}$ →scales data to unit variance
  - ϵ→ small constant (e.g., 10^{-5} (to avoid division by zero
  - *Result:*
    - Mean = 0
    - Variance = 1

- **Step 4: Scale and Shift (Learnable Parameters)**

$$y_i = \gamma \hat{x}_i + \beta$$

  - Why γ and β?
  - Normalization may limit representation power
    - γ: scale parameter
    - β: shift parameter
  - Network learns:
    - How much normalization is needed
    - Where to shift activations

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Batch Normalization

- **Formula**
  - For a mini-batch $B = \{x_1, \ldots, x_m\}$:

$$\mu_B = \frac{1}{m}\sum x_i \,, \sigma_B^2 = \frac{1}{m}\sum (x_i - \mu_B)^2 \quad \hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

- Mini-batch activations:

[2,4,6,8]

- **Mean**

$$\mu = 5$$

- **Variance**

$$\sigma^2 = \frac{\left(2-5\right)^2 + \left(4-5\right)^2 + \left(6-5\right)^2 + \left(8-5\right)^2}{4} = 5$$

Standard deviation

[2,4,6,8]

Normalized values

[−1.34,−0.45,0.45,1.34]

Assume:

$$\gamma = 2,\ \beta = 1$$

Final output

[−1.68,0.10,1.90,3.68]

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Question

- A mini-batch of neuron activations is given as:

  [2,4,6,8]

- Assume:
  - $\gamma = 1$
  - $\beta = 0$
  - $\epsilon = 0$
- **Tasks:**
  - Compute batch mean
  - Compute batch variance
  - Normalize the activations

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Answer

- **Step 1: Mean**

$$\mu = \frac{2+4+6+8}{4} = \boxed{5}$$

- **Step 2: Variance**

$$\sigma^2 = \frac{\left(2-5\right)^2 + \left(4-5\right)^2 + \left(6-5\right)^2 + \left(8-5\right)^2}{4}$$

$$= \frac{9+1+1+9}{4} = \boxed{5}$$

- **Step 3: Normalize**

$$\hat{x} = \frac{x-\mu}{\sqrt{\sigma^2}}$$

[−1.34,−0.45,0.45,1.34]

- **Final Output**

[−1.34,−0.45,0.45,1.34]

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Group Normalization

Definition

- **Group Normalization** divides feature channels into **fixed groups** and normalizes activations **within each group**, independent of batch size.

Objective

- Eliminate batch-size dependency
- Maintain stable normalization for small batches

Working Principle

- Split channels into predefined groups
- Compute mean and variance within each group
- Normalize activations group-wise
- Apply learnable γ and β per channel

Applications

- Object detection (Faster R-CNN)
- Semantic segmentation
- Medical imaging models

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Group Normalization

- **Step 1: Define Groups**
  - Assume:
    - C channels
    - G groups
  - Each group has C/G channels
- **Step 2: Compute Group Mean**

$$\mu_G = \frac{1}{|S|}\sum_{k \in S} x_k$$

  - S: set of activations in **one group**
  - Mean computed **per group**, not per batch
  - *Why?* → Works even when batch size = 1.

- **Step 3: Compute Group Variance**

$$\sigma_G^2 = \frac{1}{|S|}\sum_{k \in S}(x_k - \mu_G)^2$$

  - Same idea as Batch Norm, but restricted to group channels.
- **Step 4: Normalize and Scale**

$$y_k = \gamma \frac{x_k - \mu_G}{\sqrt{\sigma_G^2 + \epsilon}} + \beta$$

  - Normalize within each group
  - γ, β are learned per channel

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Group Normalization

- Assume:
  - Feature map with **4 channels**
  - Values:

$$[1,3,5,7]$$

- Let **2 groups**:
  - Group 1:$[1, 3]$
  - Group 2:$[5,7]$

Group 1

Mean:

$$\mu = 2$$

Variance:

$$\sigma^2 = 1$$

Normalized:

$$[-1,1]$$

Group 2

Mean:

$$\mu = 6$$

Variance:

$$\sigma^2 = 1$$

Normalized:

$$[-1,1]$$

Final Output

$$[-1,1,-1,1]$$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Instance Normalization

Definition	Objective	Working Principle	Applications
• **Instance Normalization** normalizes each sample **independently per channel**, removing instance-specific contrast and illumination information.	• Remove appearance-specific variations • Preserve content while altering style	• Compute mean and variance for each channel of a single image • Normalize feature maps independently • Optional scale and shift parameters	• Neural style transfer • Image-to-image translation (CycleGAN) • Artistic rendering

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Instance Normalization

Idea
- Divides **channels into groups**
- Normalizes **within each group**
- Independent of batch size

- **Step 1: Compute Instance Mean**

$$\mu_I = \frac{1}{HW}\sum_{h=1}^{H}\sum_{w=1}^{W} x_{hw}$$

  - H, W: height & width of feature map
  - Mean computed for **each channel of one image**
  - Removes **instance-specific contrast**

- **Step 2:** Compute Instance Variance

$$\sigma_I^2 = \frac{1}{HW}\sum_{h=1}^{H}\sum_{w=1}^{W}(x_{hw} - \mu_I)^2$$

- **Step 3: Normalize**

$$\hat{x}_{hw} = \frac{x_{hw} - \mu_I}{\sqrt{\sigma_I^2 + \epsilon}}$$

- **Step 4: Optional Scale and Shift**

$$y = \gamma\hat{x} + \beta$$

  - Often:
    - $\gamma = 1$
    - $\beta = 0$

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Instance Normalization Example

Idea

- Normalizes **each channel of each sample independently**
- No batch dependency
- Removes **instance-specific contrast information**

- Single image, one channel:

  [10,20,30,40]

- **Mean**

  $$\mu = 25$$

- **Variance**

  $$\sigma^2 = 125$$

- **Std**

  $$\sigma = 11.18$$

Normalized

[−1.34,−0.45,0.45,1.34]

Apply:

$$\gamma = 1,\ \beta = 0$$

Final output remains the same.

[−1.34,−0.45,0.45,1.34]

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Comparison

Aspect	Batch Norm	Group Norm	Instance Norm
Normalization axis	Batch	Channel groups	Per instance
Batch size dependent	Yes	No	No
Best for	Large batches	Small batches	Style transfer
CNN usage	High	Very high	Medium
Stability	Medium	High	Very high

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

THANK YOU

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.